

Relatório 23 - Reconhecimento de Emoções com TensorFlow 2.0 e Python

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre Visão Computacional aplicada ao reconhecimento de emoções. A tarefa consistia em assistir ao curso “Reconhecimento de Emoções com TensorFlow 2.0 e Python” e desenvolver uma aplicação prática.

2. Desenvolvimento

Sobre o curso, algumas coisas que houve de diferente foi sobre os call-backs, função ELU e parada antecipada, já o resto de conteúdo eu já sabia, então não houve muitas coisas de diferente.

Os call-backs, de forma geral, são funções que o Keras chama em algumas partes específicas do treino, assim, não precisa escrever um loop de treino na mão. O ModelCheckpoint, por exemplo, salva a versão do modelo com menor perda na validação, não na última epoch, isso é importante, pois, não precisa de mais epochs para um modelo melhor, já que depois de um ponto ele começa decorar o treino e piorar a validação.

ELU (Exponential Linear Unit) é uma ativação alternativa da ReLU, o ReLU zera qualquer valor negativo, o que faz uns neurônios morrer. O ELU permite uma saída negativa pequena, assim mantém o gradiente vivo mesmo para entradas negativas, basicamente, vai ajudar a rede neural convergir de forma mais estável.

Parada antecipada (*Early Stopping*), basicamente, ela monitora uma métrica (no meu caso, a `val_loss`) e interrompe o treino se ela parar de melhorar em um determinado número de épocas (a “paciência”). Isso vai ajudar a evitar o gasto de tempi e computação treinando um modelo que no final vai estar piorando a generalização. Ele paramêtros trabalha em conjunto com o ModelCheckpoint, mesmo que o treino continue depois do melhor ponto, o arquivo salvo vai ser o melhor do momento e não o último.

Uma coisa que pesquisei, que eu lembre não ouvi falar no curso, seria uma forma de “normalizar” as classes, por exemplo, a classe nojo tem poucas imagens enquanto a alegria tem 9 vezes mais que a nojo. Ela funciona em cima da distribuição dos rótulos, atribuindo um peso maior de erro para classe menor (menos imagens neste caso) na função de perda.

Sobre a parte prática, abaixo segue um fluxograma da sequência que segui para desenvolver minha parte prática.

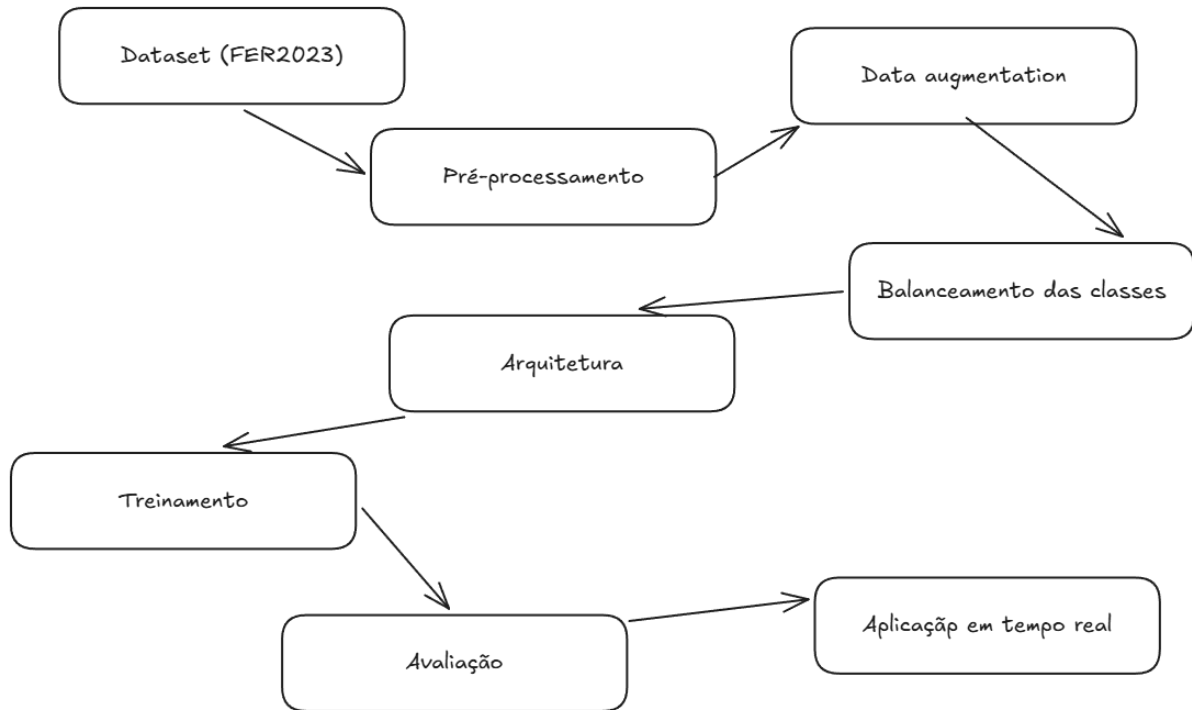


Figura 1. Fluxograma do que foi seguido para realizar a atividade prática. (autoria própria)

1. **Dataset (FER2013)**: 35.887 imagens 48×48 em escala de cinza, 7 classes
2. **Pré-processamento**: normalização (0–255 > 0–1) e divisão treino/validação/teste
3. **Balanceamento de classes**: cálculo e suavização dos pesos por classe
4. **Arquitetura**: 4 blocos convolucionais + classificador denso
5. **Data Augmentation**: pipeline tf.data com repetição infinita
6. **Treinamento**: com callbacks de redução de learning rate e parada antecipada
7. **Avaliação**: matriz de confusão e métricas por classe
8. **Aplicação em tempo real**: juntei as atividades do card 22 com este card para fazer a detecção de emoções em tempo real.

Aspecto	Escolha	Justificativa
Blocos convolucionais	4, dobrando os filtros (32→64→128→256)	Mesmo padrão da arquitetura de melhor desempenho já testada (Arquitetura 2), usado como ponto de partida
Ativação	ELU	Testada com bons resultados na Arquitetura 2; evita o problema de "neurônios

		mortos" do ReLU em valores negativos
Regularização	BatchNormalization + Dropout 0.3 por bloco	Dropout um pouco mais alto que o da Arquitetura 2 (0.2), para compensar a maior capacidade do classificador denso
Otimizador	Adam, lr inicial 0.001	Padrão já validado nas arquiteturas anteriores
Callbacks	ReduceLROnPlateau, EarlyStopping, ModelCheckpoint	Evita treinar epochs desnecessárias e garante que o modelo salvo seja o de menor loss de validação, não o da última epoch

Métrica	Peso bruto ("balanced")	Peso suavizado (raiz quadrada)
Acurácia (model.evaluate)	62,72%	65,59%
Acurácia (matriz de confusão)	64,61%	65,90%
Perda (loss)	1,0078	0,9202

Recall e precisão por classe, antes e depois da suavização:

Classe	Recall (bruto)	Recall (suavizado)	Precisão (bruto)	Precisão (suavizado)
Raiva	50,2%	55,2%	60,0%	60,2%
Nojo	76,9%	59,6%	38,1%	49,2%
Medo	39,1%	41,1%	56,5%	58,3%
Feliz	84,9%	85,7%	87,1%	87,3%
Triste	51,2%	52,2%	55,0%	55,1%
Surpreso	78,0%	75,8%	68,3%	71,0%
Neutro	72,7%	75,1%	54,7%	55,9%

A suavização trocou parte do recall de "Nojo" por precisão (esperado: o modelo para de "chutar" essa classe com tanta frequência) e, como efeito colateral positivo, melhorou a maioria das outras classes também, elas deixaram de perder amostras "roubadas" pela superprevisão de "Nojo".

A classe "Medo" permanece como o ponto mais fraco (41% de recall) mesmo após o balanceamento. A matriz de confusão mostra que ela se confunde principalmente com "Triste" e "Surpreso", uma ambiguidade visual real (traços faciais parcialmente sobrepostos entre essas expressões), não um problema de quantidade de dados, portanto não resolvida por reponderação de classes.

3. Conclusão

A prática mostrou que em *datasets* ruidosos como o FER2013, boa parte do ganho de desempenho não vem apenas da arquitetura da rede, mas de diagnosticar corretamente o que está acontecendo durante o treinamento. Já o balanceamento de classes mostrou que aplicar uma técnica corretamente não basta, já o peso "ideal" segundo a fórmula padrão pode ser ruim demais na prática, e suavizá-lo exige entender sobre escolher entre recall e precisão, não apenas copiar a função da documentação (meio que o que eu fiz a primeira fez). O resultado final mostra que o modelo está ajustado dentro do que essa arquitetura permite e que a confusão remanescente entre "Medo", "Triste" e "Surpreso" é uma limitação do problema em si, não da implementação.

4. Referências

Granatyr, J.; Alves, G.; IA Expert Academy. **Reconhecimento de Emoções com TensorFlow 2.0 e Python**. 2021. Disponível em:

<https://www.udemy.com/course/reconhecimento-emocoes-tensorflow-20-python/?couponCode=26BBPAA2MX>.

scikit-learn developers. **compute_class_weight**. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html.